

What if one single app could handle both REST APIs and Microservices at the same time?

No separate deployments... no extra complexity... just a hybrid system running everything together.



What is a Hybrid Application?



- A single NestJS application that runs:
- HTTP APIs (REST) for external clients
- Microservices (internal communication)
simultaneously



Dual-Listening Mode



- One app listens on:
- HTTP Port (e.g., 3000) → for client requests
- Microservice Transport (TCP/gRPC/Redis)
→ for internal communication
- This is called Dual-Listening Architecture



Why Hybrid Architecture?



- Combine:
- External communication (HTTP APIs)
- Internal fast communication (Microservices)
- Avoid managing multiple separate services
in early stages



HTTP Layer (Client-Facing)



- Handles requests from:
- Browser / Frontend / Postman
- Example:
- /orders
- /users



Microservice Layer



- Services communicate internally using:
- High-performance protocols like TCP or gRPC (instead of traditional REST/HTTP 1.1)
- Faster, lightweight, and efficient



Real-World Flow Example



- User hits: /create-order (HTTP request)
- Inside same app:
- Order logic runs
- Calls Payment Service (Microservice)
- Triggers Notification Service (Email/SMS in background)
- One request → multiple internal service interactions



Performance Advantage



- Internal calls:
- Faster than REST
- Smaller payload (especially gRPC)
- Efficient for service-to-service communication



When to Use Hybrid Apps?



- Early-stage products
- MVP development
- Learning microservices
- Gradual migration from monolith → microservices



Common Mistakes

